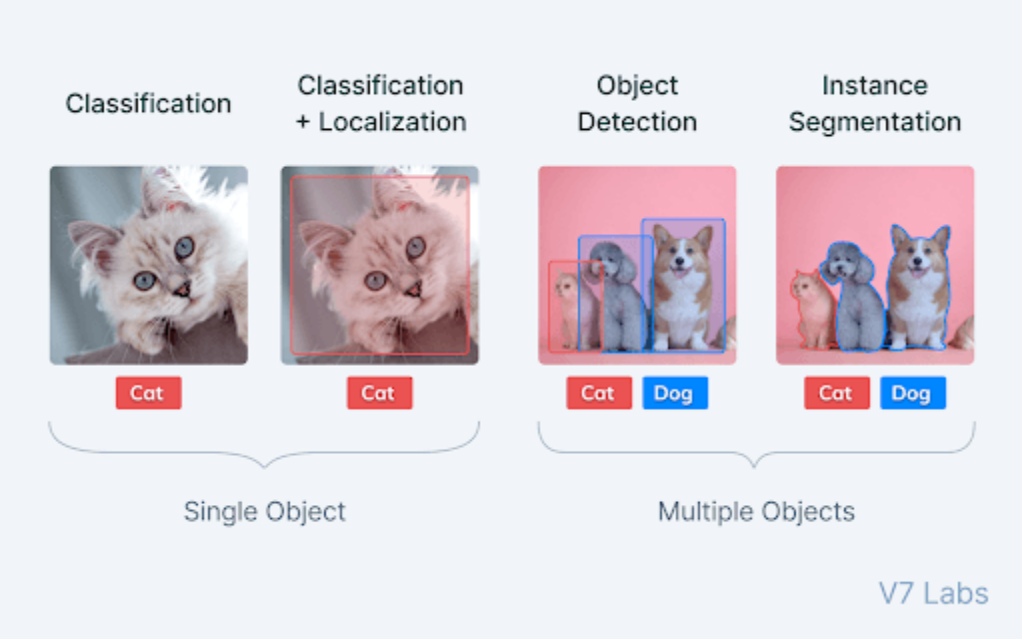
What is Image Segmentation?

Image segmentation is a sub-domain of computer vision and digital image processing which aims at grouping similar regions or segments of an image under their respective class labels.

Since the entire process is digital, a representation of the analog image in the form of pixels is available, making the task of forming segments equivalent to that of grouping pixels.

Image segmentation is an extension of image classification where, in addition to classification, we perform localization. Image segmentation thus is a superset of image classification with the model pinpointing where a corresponding object is present by outlining the object's boundary.



Types of Image Segmentation tasks

Image segmentation tasks can be classified into three groups based on the amount and type of information they convey.

While semantic segmentation segments out a broad boundary of objects belonging to a particular class, instance segmentation provides a segment map for each object it views in the image, without any idea of the class the object belongs to.

Panoptic segmentation is by far the most informative, being the conjugation of instance and semantic segmentation tasks. Panoptic segmentation gives us the segment maps of all the objects of any particular class present in the image.

Let’s explore these tasks in greater detail.

Semantic segmentation

Semantic segmentation refers to the classification of pixels in an image into semantic classes. Pixels belonging to a particular class are simply classified to that class with no other information or context taken into consideration.

As might be expected, it is a poorly defined problem statement when there are closely grouped multiple instances of the same class in the image. An image of a crowd in a street would have a semantic segmentation model predict the entire crowd region as belonging to the "pedestrian" class, thus providing very little in-depth detail or information on the image.

Instance segmentation

Instance segmentation models classify pixels into categories on the basis of "instances" rather than classes.

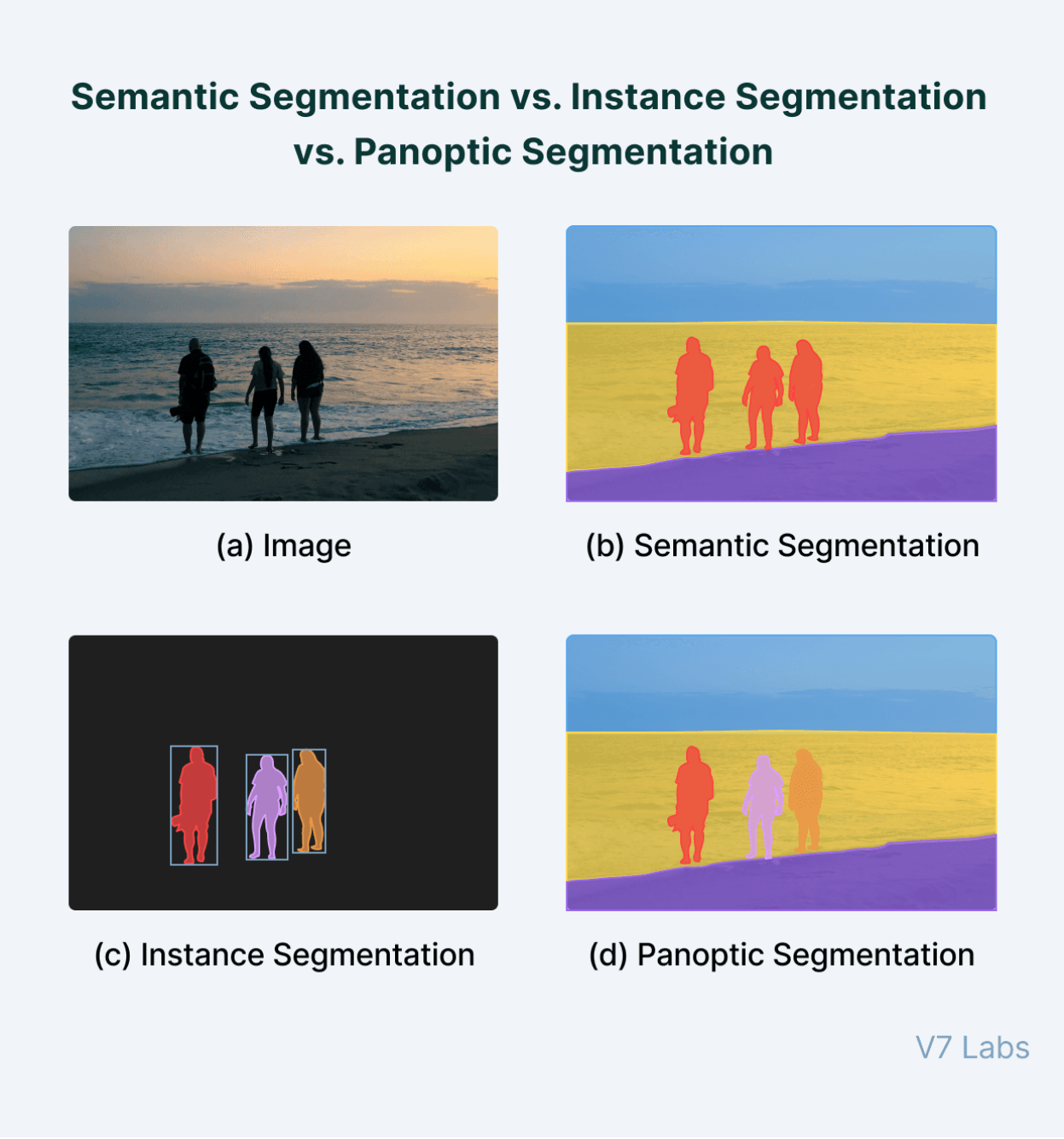
An instance segmentation algorithm has no idea of the class a classified region belongs to but can segregate overlapping or very similar object regions on the basis of their boundaries.

If the same image of a crowd we talked about before is fed to an instance segmentation model, the model would be able to segregate each person from the crowd as well as the surrounding objects (ideally), but would not be able to predict what each region/object is an instance of.

Panoptic segmentation

Panoptic segmentation, the most recently developed segmentation task, can be expressed as the combination of semantic segmentation and instance segmentation where each instance of an object in the image is segregated and the object's identity is predicted.

Panoptic segmentation algorithms find large-scale applicability in popular tasks like self-driving cars where a huge amount of information about the immediate surroundings must be captured with the help of a stream of images.



Traditional Image Segmentation techniques

Image segmentation originally started from Digital Image Processing coupled with optimization algorithms. These primitive algorithms made use of methods like region growing and snakes algorithm where they set up initial regions and the algorithm compared pixel values to gain an idea of the segment map.

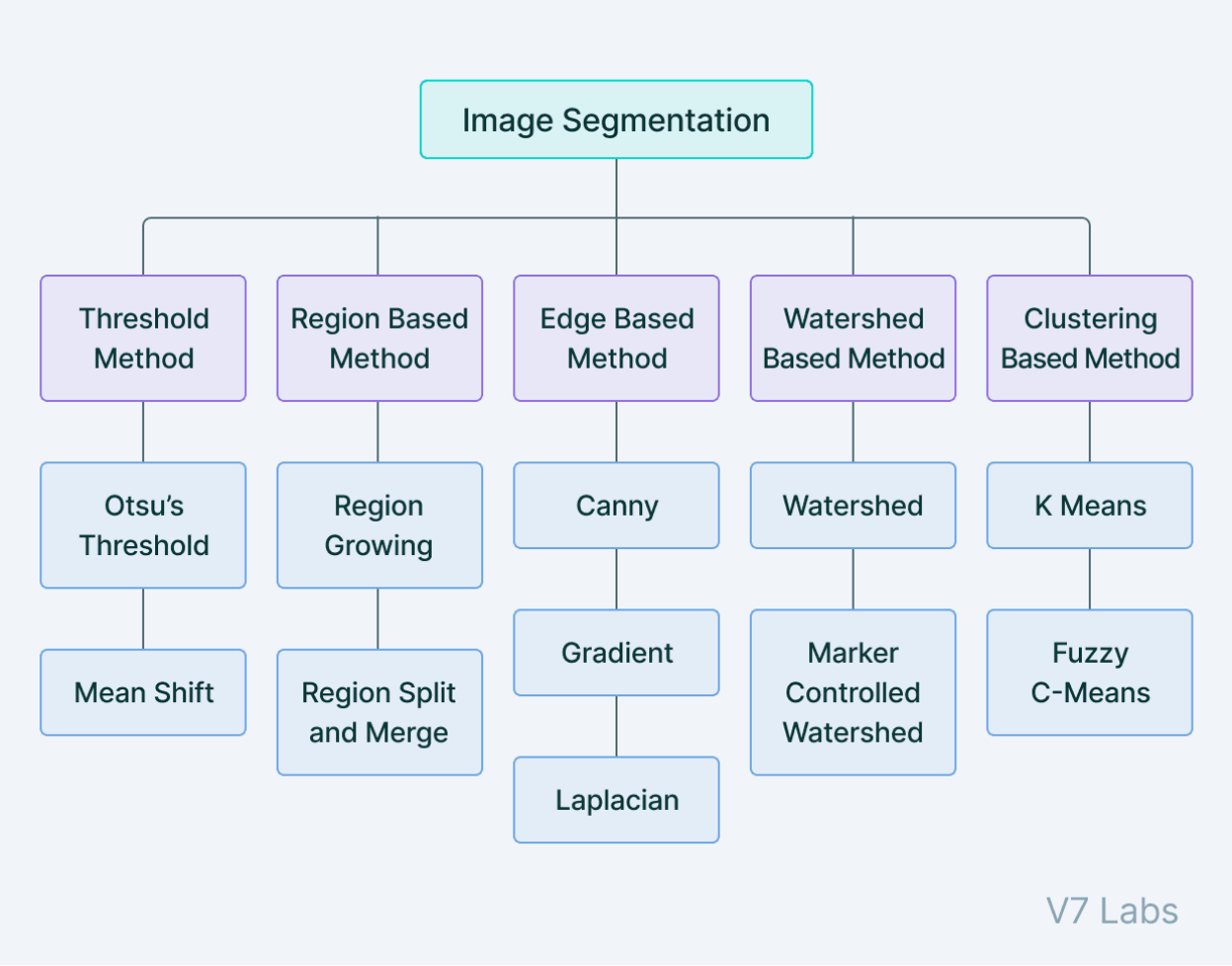


Image Segmentation
Threshold Method
Region Based Method
Edge Based Method
Watershed Based Method
Clustering Based Method
Otsu's Threshold
Region Growing
Canny
Watershed
K Means
Mean Shift
Region Split and Merge
Gradient
Marker Controlled Watershed
Fuzzy C-Means
Laplacian
V7 Labs

An overview of Semantic Image Segmentation

Input

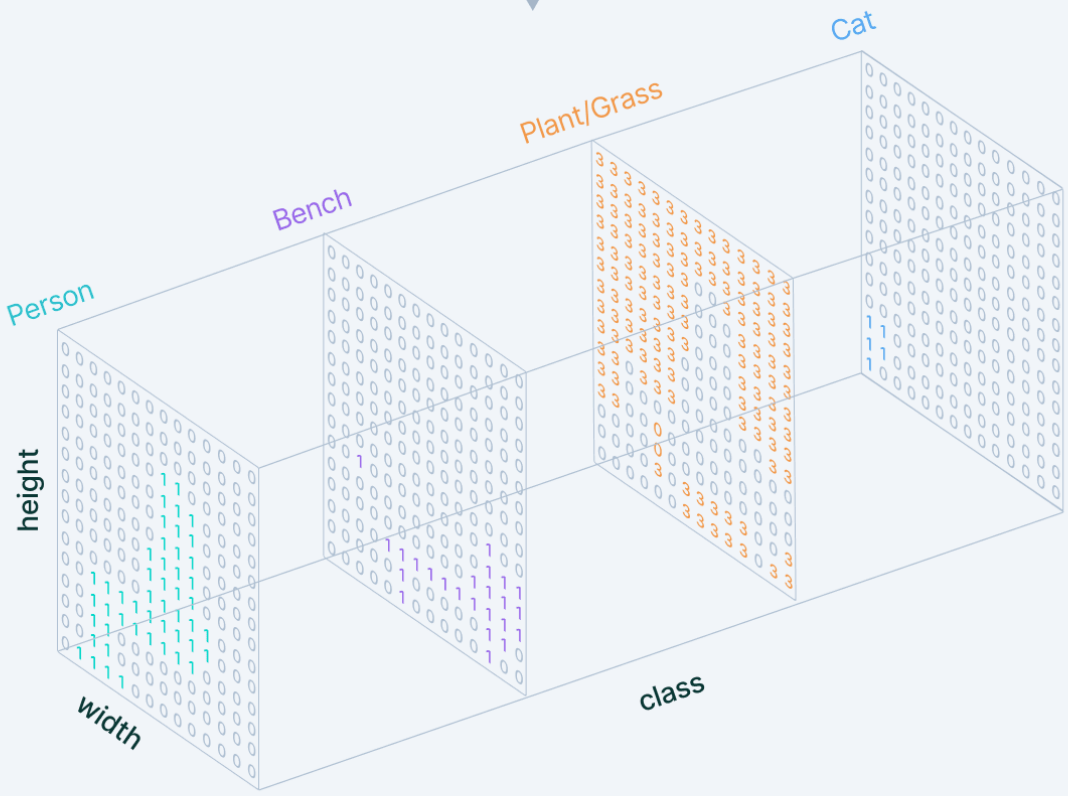
Segmented

1: Person
2: Bench
3: Plant/Grass
4: Cat

3 3 3 3 3 3 3 3 3 3 3 3 3 3
3 3 3 3 3 3 3 3 3 3 3 3 3 3
3 3 3 3 3 3 3 3 3 3 3 3 3 3
3 3 3 3 3 3 3 3 1 1 3 3 3 3
3 3 3 3 3 3 3 3 1 1 1 3 3 3
3 3 3 3 3 3 3 3 1 1 1 3 3 3
3 3 3 3 3 3 3 3 1 1 1 3 3 3
3 3 3 3 3 3 3 1 1 1 1 3 3 3
3 3 3 3 3 3 3 1 1 1 1 3 3 3
3 3 2 3 3 3 1 1 1 1 1 2 3 3
3 3 1 1 1 1 1 1 1 1 1 2 2 2
3 3 1 1 1 1 1 1 1 1 2 2 2 2
4 4 1 1 2 2 2 2 2 2 2 2 2 2
4 4 1 1 3 2 3 3 3 3 3 2 2 3
4 1 1 1 1 2 3 3 3 3 3 2 3 3

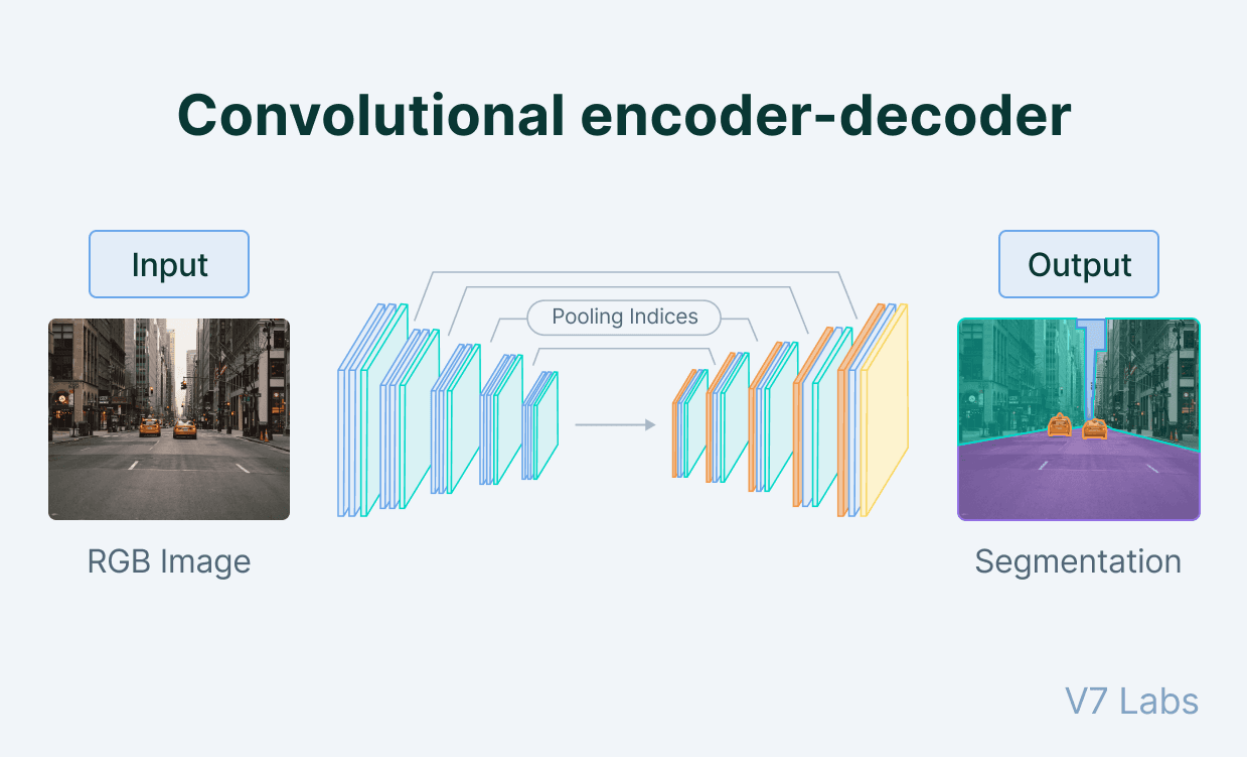
Semantic Labels

One-hot Encoding

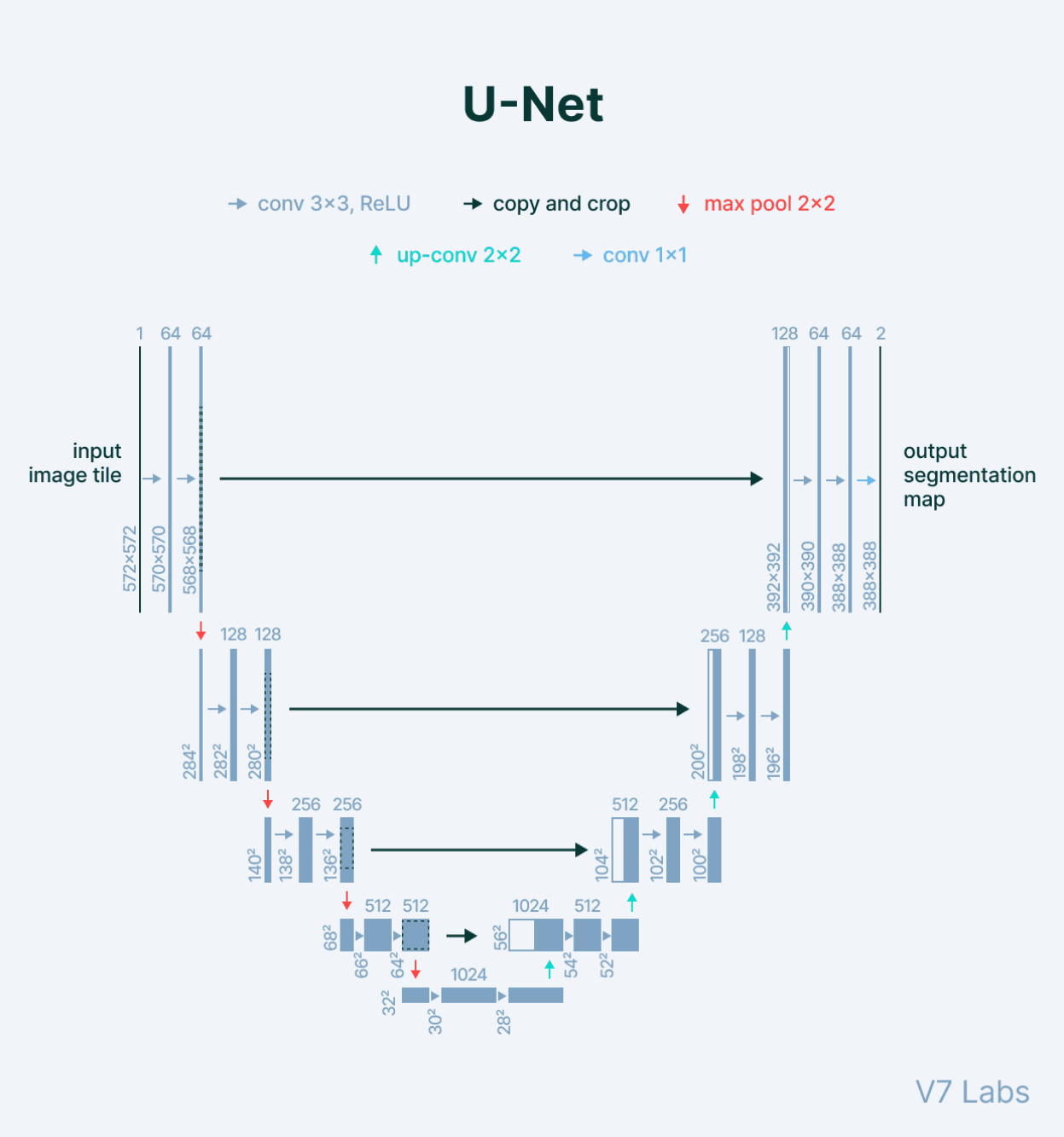


Convolutional Encoder-Decoder Architecture

Encoder decoder architectures for semantic segmentation became popular with the onset of works like SegNet (by Badrinarayanan *et. a.*) in 2015



SegNet was accompanied by the release of another independent segmentation work at the same time, U-Net (by Ronnerberger *et. al.*), which first introduced skip connections in Deep Learning as a solution for the loss of information observed in downsampling layers of typical encoder-decoder networks.



The Problem: Reconstructing the Image

In the encoder (downsampling) part of U-Net, the image goes through pooling layers and convolutions that **reduce the spatial dimensions** (height and width) while increasing the depth (number of features). For example, a 256×256 image might become 16×16 at the bottleneck.

The decoder (upsampling) part needs to **reconstruct the original image size** to produce a segmentation map where each pixel is classified. This is where transposed convolution comes in.

What is Transposed Convolution?

Transposed convolution (also called deconvolution or upconvolution) is an operation that **increases the spatial dimensions** of a feature map while learning how to do this intelligently.

Think of it as:

- **Regular convolution:** Takes a large image → produces a smaller feature map
- **Transposed convolution:** Takes a small feature map → produces a larger reconstruction

How It Works in U-Net

In U-Net, transposed convolution is used in the **expanding path** (the right side of the "U"):

Input: 64×64×512 (small, deep feature map)
↓ Transposed Convolution (2× upsampling)
Output: 128×128×256 (larger, shallower feature map)

The Mechanics:

1. **Upsampling**: A 2×2 transposed convolution with stride 2 will double the height and width
2. **Feature Learning**: Unlike simple interpolation, it learns the optimal way to upsample through trainable weights
3. **Skip Connections**: The upsampled result is then concatenated with the corresponding feature map from the encoder path

Simple Example

Imagine a 2×2 input:

[[1, 2],
[3, 4]]

With a 3×3 kernel and stride 2, transposed convolution would produce a 4×4 output by:

- Placing the kernel at each input position
- Multiplying kernel weights by the input value
- Summing overlapping regions

Why Not Use Simple Interpolation?

You might ask: "Why not just use bilinear or nearest-neighbor upsampling?"

Answer: Transposed convolution **learns** the upsampling, while interpolation uses **fixed rules**.